Heart Disease Prediction End-to-End MLOps System

OVERVIEW

Heart Disease Prediction project turns a binary classifier into a tool that checks for heart disease using the UCI Processed Cleveland dataset. Heart Disease Prediction focus is not on how well the model predicts; Heart Disease Prediction also brings together data quality checks reproducible model selection, experiment tracking, packaging, automated validation, containerized serving, Kubernetes deployment and request-level monitoring into a single traceable MLOps demonstration.

Project Outcomes:

- Dataset: 303 records / 13 features
- Model Selection: Tuned Random Forest
- Final Test ROC-AUC: 0.9556 (61 test cases)
- CI: 3 passing jobs

Objectives and assessed capabilities:

- Prepare the source dataset inspect feature distributions and class balance and build a train/test workflow.
- Compare four model families under five-fold cross-validation; choose the best candidate before looking at the held-out test set.
- Track metrics in MLflow package the fitted pipeline and register a versioned champion model.
- Validate source changes through GitHub Actions build a Docker image serve predictions on K3s and watch API health and latency.

System Boundary:

The repository contains scripts and a CI workflow. The deployed API uses a model from a MLflow registry, CI does not push a new model or deploy containers. Kubernetes and the Load Balancer stay local to the development environment and are not publicly hosted.

Principal Reported Evidence:

Evaluation	Operations
5-fold CV selected ROC-AUC: 0.9044	GitHub Actions run #7: success (2m 11s)
Held-out accuracy: 88.52%, Recall: 92.86%	22 automated tests; lint, train, Docker validation
61 test cases, TN: 28, FP: 5, FN: 2, TP: 26	200 sequential requests, p95 latency 51.32ms

DATASET AND EXPLORATORY ANALYSIS

Data Acquisition & EDA:

The raw Cleveland file from the UCI Heart Disease collection is in the repository and has been turned into a binary-classification table. Original diagnosis value 0 means “no disease”, and values 1- 4 mean “disease present”. The target column is now called heart_disease.

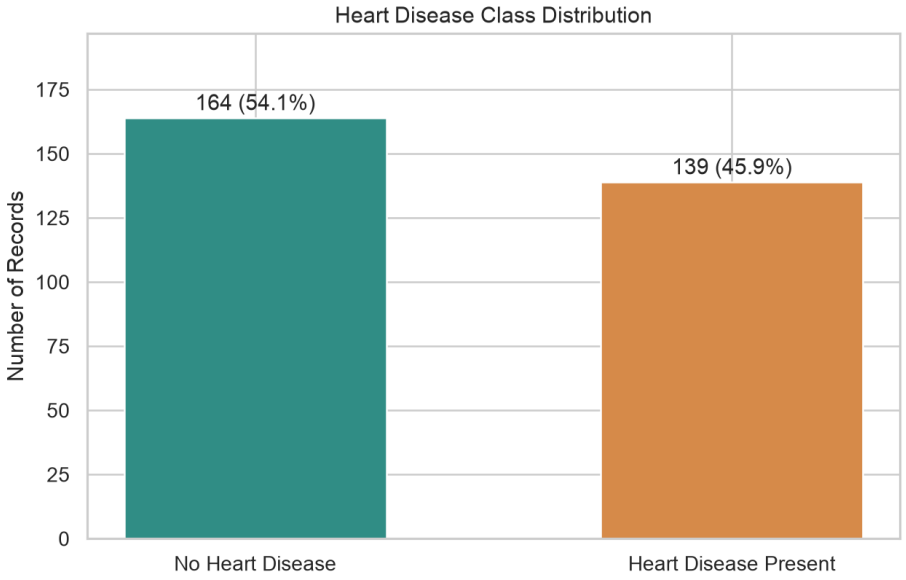


Fig 1. Class distribution in the Cleveland data.

Exploratory observations:

The outcome classes are almost balanced. The sample is small. Feature histograms show ranges and shapes for numeric measurements such as age resting blood pressure, serum cholesterol, maximum heart rate and ST depression. These checks led to the decision to keep preprocessing inside the modeling pipeline than apply it to the entire dataset before splitting.

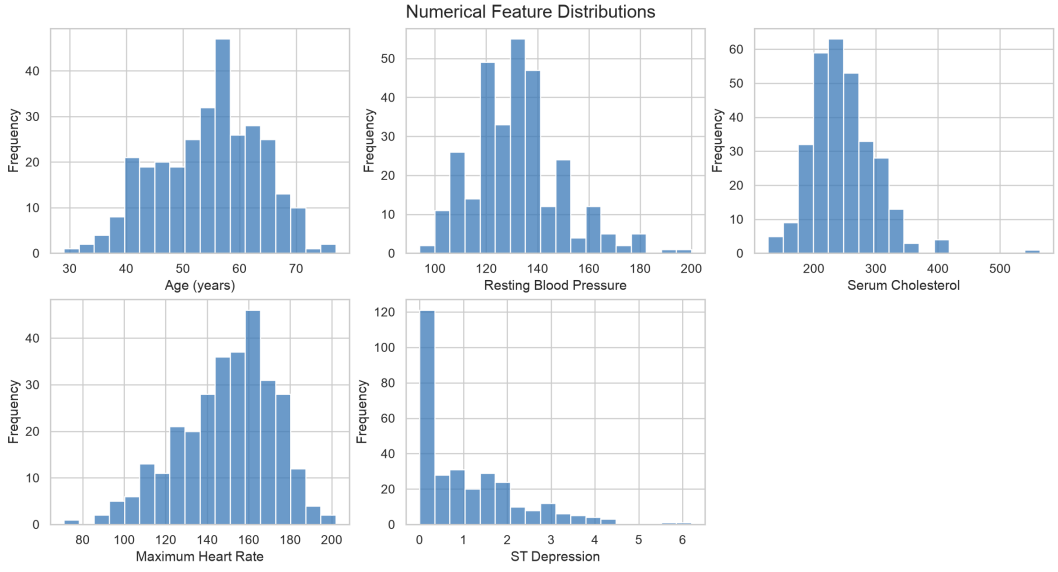


Fig 2. Representative numeric-feature distributions (existing EDA output).

EXPERIMENTAL DESIGN

Feature Engineering & Leakage Control:

One held-out test set, five-fold cross-validation limited to training records.

STAGE	RECORDS	ROLE
Raw / prepared data	303	13 features + binary target
Stratified training portion	242 (80%)	Cross-validation and hyperparameter tuning
Stratified held-out portion	61 (20%)	One final evaluation after model choice

All splits use random_state=42. The labels are stratified so that the train and test partitions keep the class mix about the same. Predictor columns include age, sex, chest-pain type, blood pressure, cholesterol, fasting glucose resting ECG, heart rate, exercise-induced angina, oldpeak, slope, major-vessel count and thalassemia-related status.

Preprocessing protocol:

- The source data has missing markers; preprocessing is fitted on training folds inside scikit-learn Pipelines.
- During cross-validation each fold training data set the preprocessing parameters; the validation folds are transformed only with those fitted parameters.
- The chosen pipeline is refitted on all 242 training records before the test-set assessment.

Correlation inspection:

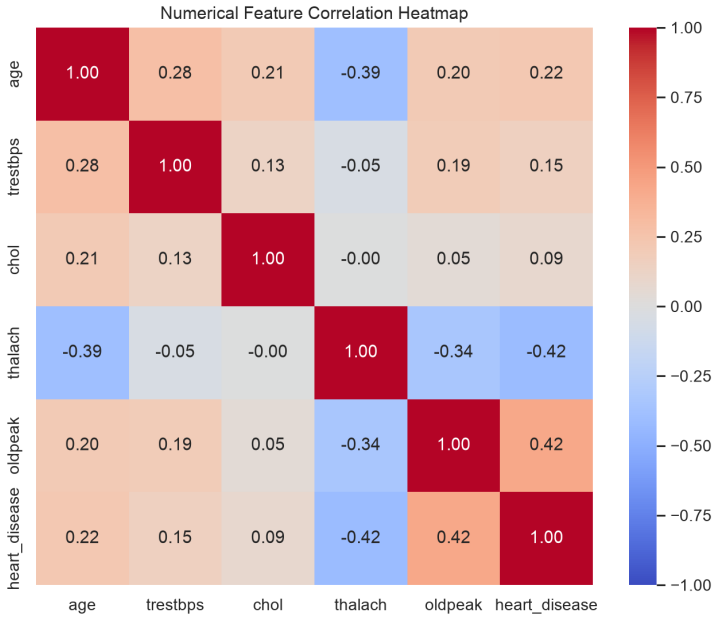


Fig 3. Correlations of selected features, from the EDA.

Correlation patterns give exploratory context, not causal links or proof of clinical relevance. Hyperparameter optimization and model choice use mean cross-validation ROC-AUC; the held-out test subset is kept out of tuning.

MODELING AND OPTIMIZATION

Model Comparison & Tuning:

I have tested eight setups from four main classifier families using average metrics from five-fold training cross-validation.

MODEL	VARIANT	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
Logistic Regression	Baseline	0.8471	0.8768	0.7834	0.8245	0.9025
Logistic Regression	Tuned	0.8304	0.8232	0.8099	0.8132	0.9028
Random Forest	Baseline	0.8139	0.8129	0.7739	0.7900	0.8923
Random Forest	Tuned	0.8056	0.7965	0.7826	0.7854	0.9044
SVM (RBF)	Baseline	0.8139	0.8303	0.7648	0.7893	0.8884
SVM (RBF)	Tuned	0.8139	0.8303	0.7648	0.7893	0.8884
Gradient Boosting	Baseline	0.7975	0.7724	0.7925	0.7809	0.8563
Gradient Boosting	Tuned	0.8469	0.8777	0.7830	0.8234	0.8958

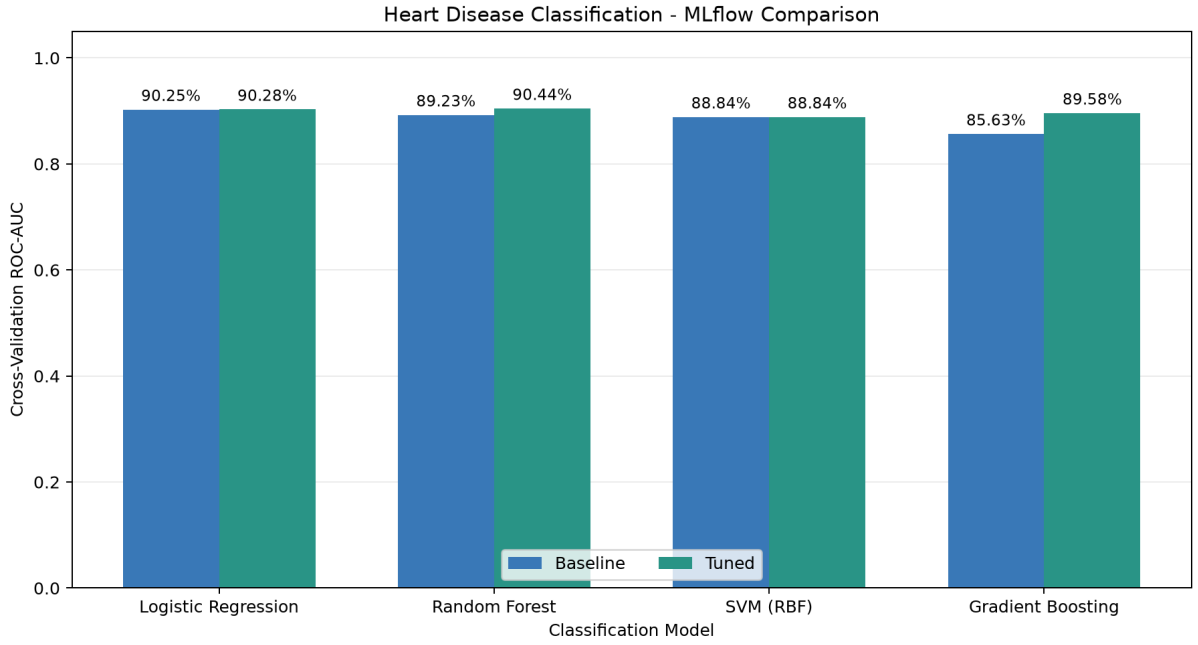


Fig 4. Baseline vs Tuned mean Cross-Validation ROC-AUC for each model family.

Selected HyperParameters and Rationale:

Logistic Regression: C = 0.1, class_weight = balanced

Random Forest: 200 trees, max_depth = 5, min_samples_leaf = 3, class_weight = balanced

The tuned Random Forest reached the highest mean cross-validation ROC-AUC (0.9044 with a standard deviation of 0.0260). It was narrowly ahead of the tuned Logistic Regression, which had a ROC-AUC of 0.9028. The difference of 0.0016 does not prove a significant advantage. However Logistic Regression produced an accuracy and a better F1-score in the cross-validation results.

FINAL EVALUATION

I evaluated the chosen Random Forest on 61 records that were set aside before model selection.

- Accuracy: **88.52%**
- Precision: **83.87%**
- Recall: **92.86%**
- F1-Score: **88.14%**
- ROC-AUC: **95.56%**

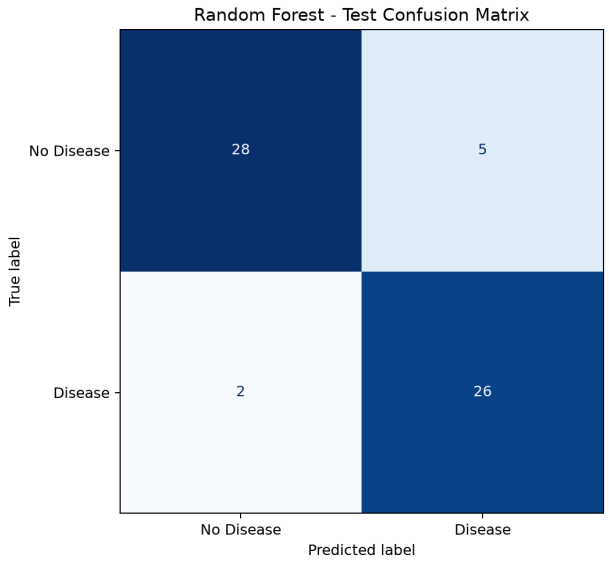


Fig 5. Confusion matrix for 61 cases.

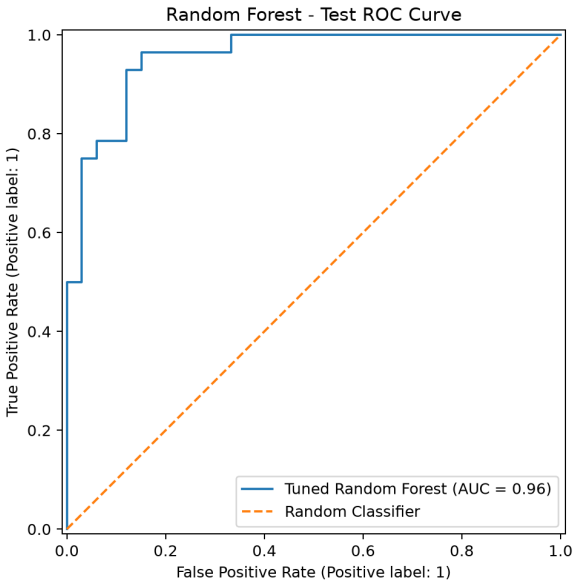


Fig 6. ROC curve for the held-out test data.

Confusion Matrix Interpretation:

ACTUAL / PREDICTED	No Disease (0)	Disease (1)
No Disease (0)	28 TN	5 FP
Disease (1)	2 FN	26 TP

Out of 28 test cases that actually had the disease, the Random Forest correctly identified 26 giving a recall of 92.86%. It missed two cases and produced five false positives among the 33 negative cases. The high-test ROC-AUC shows that the model ranks predictions well for this sample but the exact number could change with other data or different splits.

Evaluation Integrity:

I selected the initial tuned Random Forest by using cross-validation on the training set. Then tested it only once on the held-out portion. Later I added models from the four families but I did not use the original test score to rank the new table. I did not report any confidence intervals or conduct external clinical validation.

TRACKING & REGISTRY

MLflow Experiment Management:

I tracked each run packaged the models reproducibly and registered a champion alias. MLflow keeps a record of each model configuration and its cross-validation metrics. The experiment named heart-disease-model-comparison holds eight tracked configurations. I exported the comparison reports to CSV and Markdown and can be traced using MLflow run identifiers.

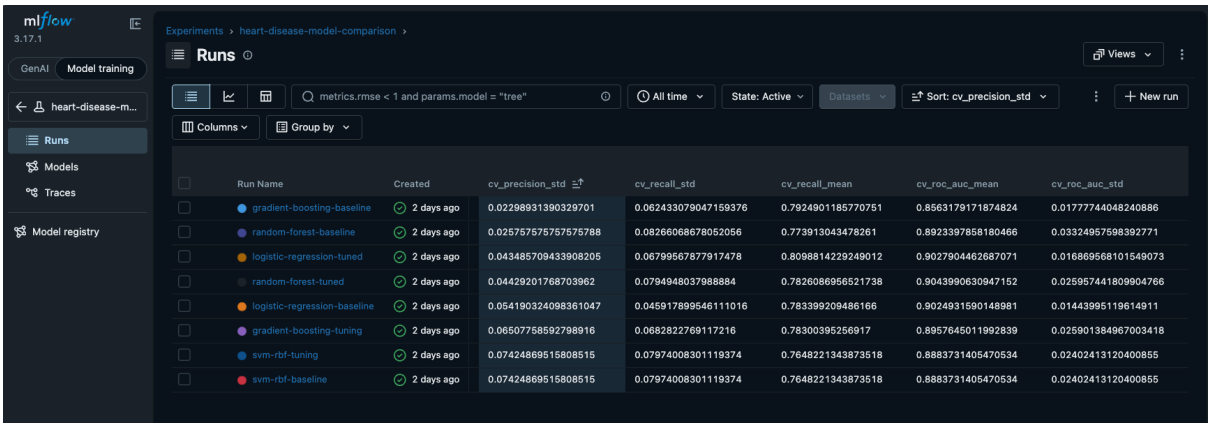


Fig 7. MLflow experiment runs showing ROC-AUC columns.

Model Packaging and Approval:

I refit the chosen tuned Random Forest inside a scikit-learn Pipeline that includes preprocessing. I log it as an MLflow model using **skops serialization** and attached a model signature and sample input. The registry now holds heart-disease-classifier, version 1 with the alias champion. The API loads **models:/heart-disease-classifier@champion** at startup.

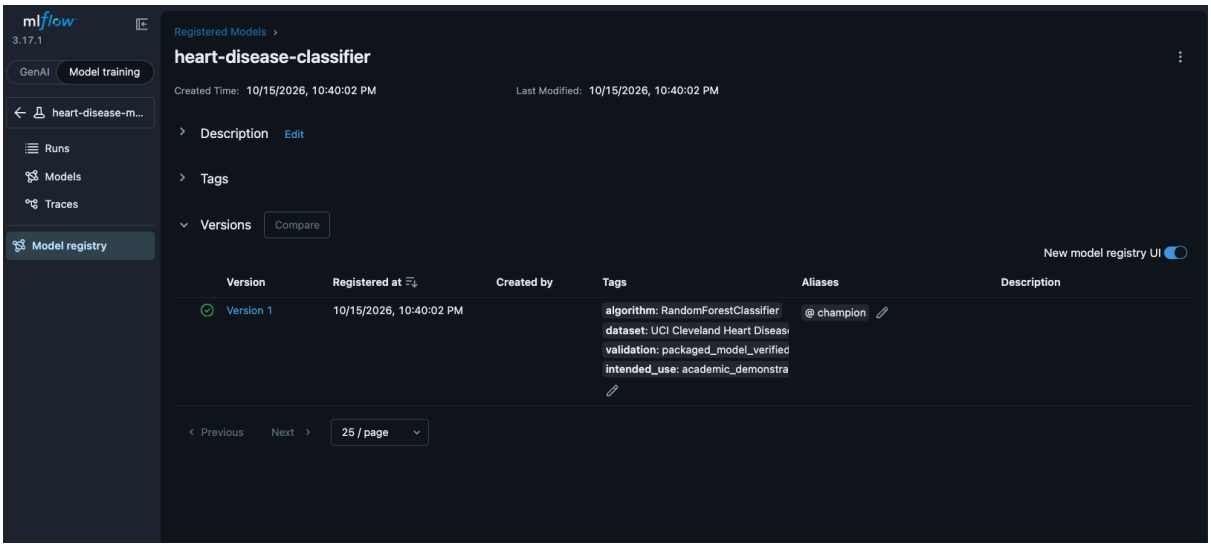


Fig 8. Registered Random Forest version 1 and champion alias.

Reproducibility boundary: The registration and verification scripts point to the original model and run IDs. A fresh clone will not reproduce the MLflow database logged artifacts or the champion alias without regenerating the artifacts and updating the identifiers.

CONTINUOUS INTEGRATION

Automated CI Validation:

The workflow triggers on pushes and pull requests to the main branch. It runs three jobs on GitHub-hosted Ubuntu: one validates the code another repeats the baseline cross-validation training and the third builds the API container. If any job fails the following Docker validation job does not finish.

Job	Activities	Observed Duration
Python Tests	Ruff static checks, Pytest (22 tests)	40 seconds
Model Training	Baseline 5-fold CV, saved console log	39 seconds
Docker Build	Build container image from Dockerfile	42 seconds

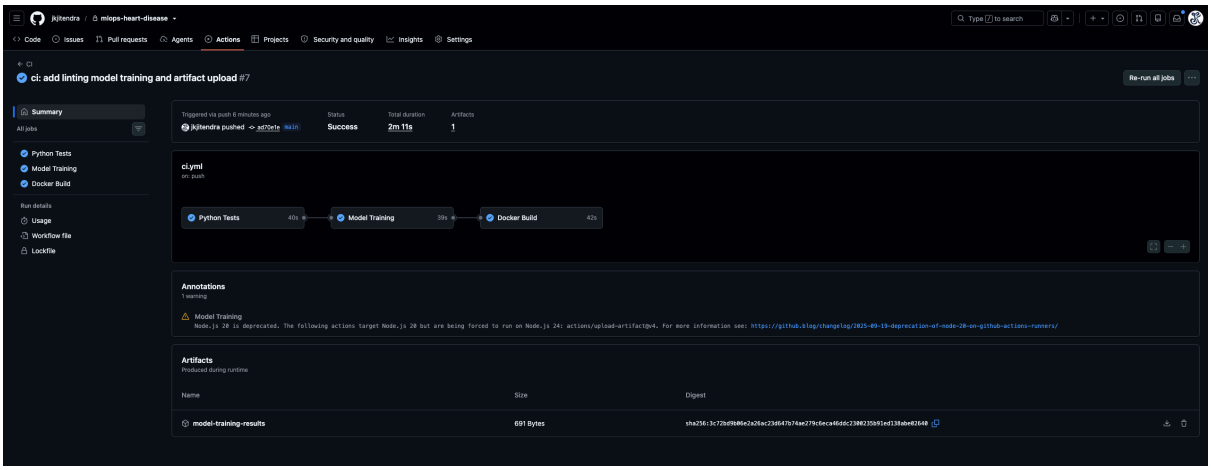


Fig 9. Successful pipeline and downloadable model-training-results artifact.

Audit Trail and Practical Meaning:

- The training job writes the log file at reports/ci/model-training.log and uploads it under model-training-results.
- The run completed successfully in 2m 11s and produced one downloadable artifact.
- Docker is built as a CI validation step. This run does not publish the image to a registry.
- There is no automatic Kubernetes rollout or automatic MLflow champion promotion in this workflow.

CONTAINERIZED DEPLOYMENT

Docker, Kubernetes & Architecture:

Containerized FastAPI inference deployed locally using Colima and K3s

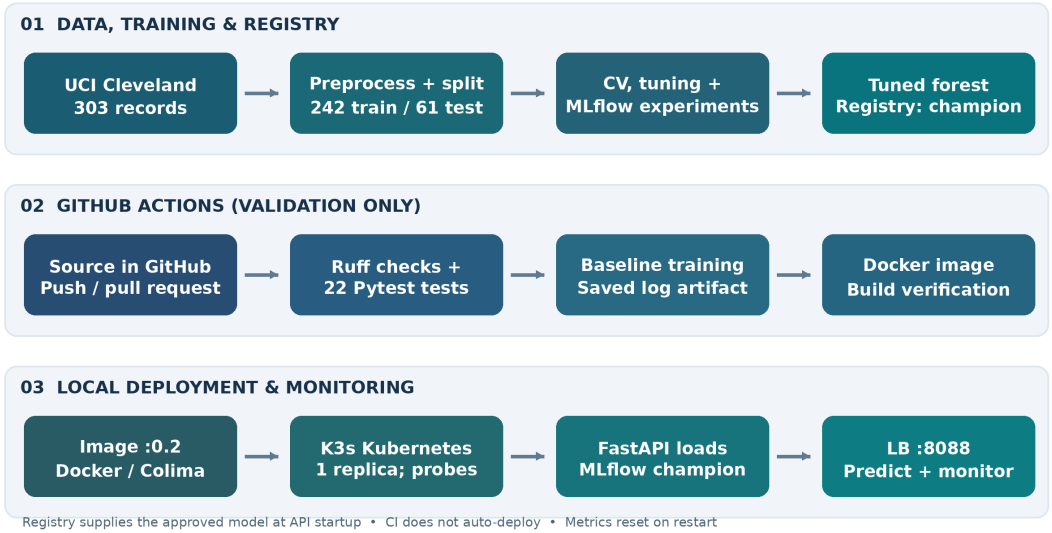


Fig 10. Simplified implementation architecture (based on docs/architecture.md).

Deployment configuration:

COMPONENT	IMPLEMENTED CONFIGURATION
Docker image	heart-disease-api:0.2, Python 3.12 slim, non-root app user
Kubernetes	Namespace heart-disease, Deployment with one replica
Probes	Startup + readiness /ready, liveness /health
Resources	CPU request 100m / limit 750m, memory 256Mi / 1Gi
Service	Local Load Balancer: 127.0.0.1:8088 -> Pod port 8000

The container connects to the MLflow host at **http://host.lima.internal:5050**. The Kubernetes manifest sets imagePullPolicy to Never so the named image must already exist in the Colima container runtime. The deployment is local and requires the model registry to be accessible when a new application Pod starts.

```
(/Volumes/MLOpsWorkspace/mlops-heart-disease/.venv) jk~$echo "=== KUBERNETES DEPLOYMENT ==="
kubectl --context colima-mlops get deployments,pods,services \
  -n heart-disease

echo "=== LOADBALANCER CONTROLLER ==="
kubectl --context colima-mlops get pods \
  -n kube-system | grep -E 'NAME|svclb-heart-disease-api-lb'
=== KUBERNETES DEPLOYMENT ===
NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api    1/1     1            1           21h

NAME                                     READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-d6b58db9-rwrd8     1/1     Running   0          9m52s

NAME                           TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api      ClusterIP      10.43.220.153   <none>        8000/TCP         21h
service/heart-disease-api-lb   LoadBalancer   10.43.199.136   192.168.5.1   8088:30407/TCP   21h
=== LOADBALANCER CONTROLLER ===
NAME                                       READY   STATUS    RESTARTS      AGE
svclb-heart-disease-api-lb-7e2bf37c-ln5fg   1/1     Running   1 (21h ago)   21h
```

Fig 11. Kubernetes Deployment, Pod and Load Balancer evidence.

OBSERVABILITY AND OPERATIONS

API Monitoring, Logging & Reliability:

FastAPI exposes the endpoints **/health**, **/ready**, **/model-info**, **/predict**, **/metrics** and **/monitor**. The dashboard shows request counts, HTTP 4xx/5xx outcomes, mean latency and p95 latency. Request logs stay available in Kubernetes. Metrics are in-process and reset when the pod restarts.

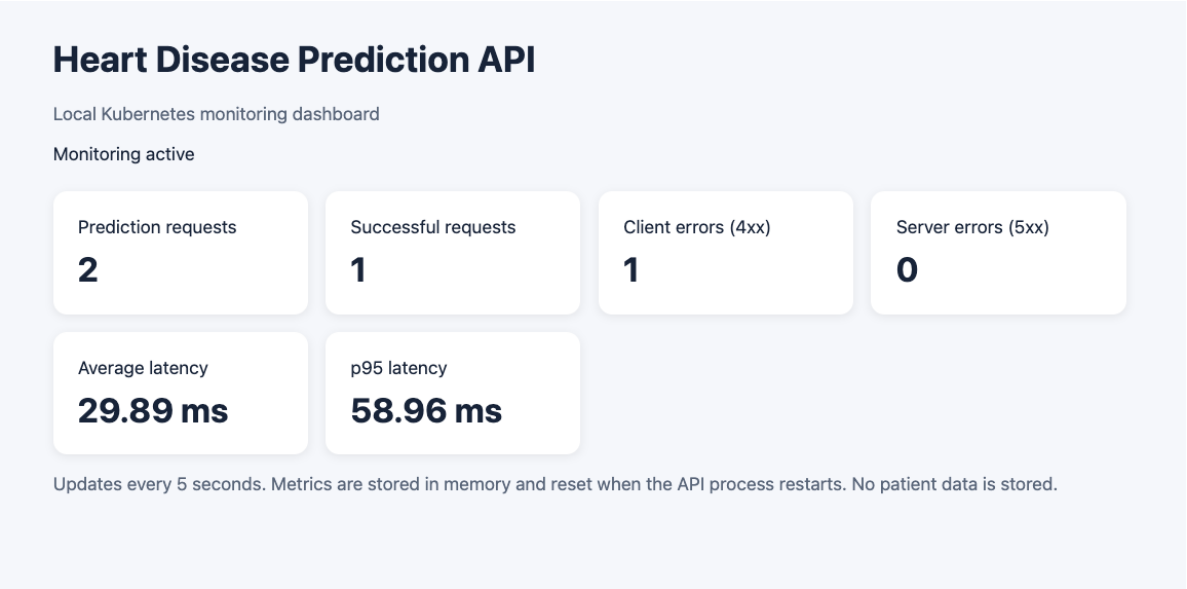


Fig 12. API monitoring interface

Latency and validity tests:

TEST / METRIC	RECORDED RESULT
200 sequential valid predictions after five warm-ups	200/200 succeeded, average latency 27.93ms
Latency distribution	p50: 22.75ms, p95: 51.32ms, p99: 56.14ms
Additional request-validation test	100 valid requests: HTTP 200, 10 invalids: HTTP 422
Resource snapshot after reliability test	62m CPU, 369 MiB memory, Pod 1/1 Ready

MLflow outage exercise:

An already running Pod kept responding even when the registry was temporarily unavailable because its model was already loaded. In a separate exercise, a newly created Pod stayed at 0/1 Ready for about 42 seconds when MLflow was missing. This shows a startup dependency under these conditions and does not prove a permanent failure.

Interpretation limit: all latency requests were sequential and made through a local Load Balancer. The numbers do not reflect traffic or a production guarantee.

REPRODUCIBILITY AND CONCLUSIONS

Reproducibility, Limits & Conclusions:

ACTION	COMMAND / LOCATION
Rebuild prepared data	python -m scripts.prepare_data
Run baseline cross-validation	python -m src.train
Run automated tests	python -m pytest -q
Run Python linting	python -m ruff check src tests scripts
Build serving container	docker build -t heart-disease-api:0.2.
Read deployment and architecture	k8s/heart-disease-api.yaml | docs/architecture.md

Limitations:

- Only 303 Cleveland records are used. The held-out test contains 61 observations and does not include external validation or confidence intervals.
- The winner's cross-validation advantage over Logistic Regression is small so the tuned-model ranking should not be overinterpreted.
- The MLflow registry and artifacts are outside the version-controlled files and some registration scripts use run-specific identifiers.
- CI runs tests, training and build validation and does not promote models, publish images or deploy to Kubernetes.
- The Kubernetes endpoint is local. Monitoring counters are temporary and latency tests exclude concurrent traffic.

Conclusion:

The submission shows the core MLOps lifecycle. It includes documented data preparation, leakage-aware evaluation, traceable MLflow experiments, versioned model packaging, automated code and training checks, containerized API delivery Kubernetes readiness management and observable request behavior. The selected tuned Random Forest achieved a mean cross-validation ROC-AUC of 0.9044 and a held-out test ROC-AUC of 0.9556 on 61 cases.

Sources and Submission Evidence:

Dataset: UCI Machine Learning Repository, Heart Disease (Cleveland), https://archive.ics.uci.edu/dataset/45/heart+disease
Repository: https://github.com/jkjitendra/mlops-heart-disease
Project evidence: The repository contains the source code, model evaluation results, MLflow experiment reports, CI/CD configuration, deployment architecture, monitoring results, and supporting screenshots. Key files include: reports/model_selection.md, reports/test_metrics.json, reports/mlflow_experiment_summary.md, docs/monitoring_baseline.md, docs/screenshots/, .github/workflows/ci.yml, docs/architecture.md
Demonstration Video: A short demonstration video link can be accessed from drive Video_Demonstration_Drive_Link (https://drive.google.com/file/d/1aTfFwcSTxKQ8tIBuP9croGbyMqDBxm7u/view?usp=sharing).
Final Report: The assignment report is provided in Microsoft Word (.docx) and PDF format under submission/.